MongoDB Practical Assignment

Subtitle: Database Creation, CRUD Operations, Query Operators and Library Management System

DATABASE CREATION

```
Microsoft Windows [Version 10.0.26300.9711]
(c) Microsoft Corporation. All rights reserved.

C:\Users\pjeff>mongosh
Current Mongosh Log ID: 6ae445db3d4d3f93c68705b6
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.14.0
Using MongoDB:          9.0.4
Using Mongosh:          2.14.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-10-28T16:21:43.110+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
------

test> use studentDB
switched to db studentDB
studentDB> db.createCollection("students")
{ ok: 1 }
studentDB> show collections
students
studentDB>
```

CRUD Operations in MongoDB

InsertOne and InsertMany():

```
studentDB> db.students.insertOne({
...     name: "Arun",
...     age: 21,
...     course: "MERN Stack",
...     status: "enrolled"
... })
{
  acknowledged: true,
  insertedId: ObjectId('6ae4467733d4d3f93c687057')
}
studentDB> db.students.insertMany([
...     {
...         name: "Jefferson",
...         age: 21,
...         course: "MERN Stack",
...         status: "enrolled"
...     },
...     {
...         name: "Priya",
...         age: 22,
...         course: "Python",
...         status: "completed"
...     },
...     {
...         name: "Rahul",
...         age: 19,
...         course: "MERN Stack",
...         status: "enrolled"
...     },
...     {
...         name: "Divya",
...         age: 20,
...         course: "Java",
...         status: "completed"
...     }
... ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ae4467f3d4d3f93c68705b6'),
    '1': ObjectId('6ae4467f3d4d3f93c68705b7'),
    '2': ObjectId('6ae4467f3d4d3f93c68705b8'),
    '3': ObjectId('6ae4467f3d4d3f93c68705b9')
  }
}
```

Find( ):

```
studentDB> db.students.find({ course: "MERN Stack" })
[
  {
    _id: ObjectId('6ae4b87f3d4d3f93c687985b'),
    name: 'Arun',
    age: 21,
    course: 'MERN Stack',
    status: 'enrolled'
  },
  {
    _id: ObjectId('6ae4b87f3d4d3f93c687985d'),
    name: 'Jefferson',
    age: 21,
    course: 'MERN Stack',
    status: 'enrolled'
  },
  {
    _id: ObjectId('6ae4b87f3d4d3f93c687985e'),
    name: 'Rahul',
    age: 19,
    course: 'MERN Stack',
    status: 'enrolled'
  }
]
studentDB> db.students.find({ status: "completed" })
[
  {
    _id: ObjectId('6ae4b87f3d4d3f93c687985c'),
    name: 'Priya',
    age: 22,
    course: 'Python',
    status: 'completed'
  },
  {
    _id: ObjectId('6ae4b87f3d4d3f93c687985f'),
    name: 'Divya',
    age: 24,
    course: 'Java',
    status: 'completed'
  }
]
studentDB>
```

updateOne and updateMany( ):

```
mongosh mongodb://127.0.0.
]
studentDB> db.students.updateOne(
...     { name: "Arun" },
...     { $set: { status: "completed" } }
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
studentDB> db.students.updateMany(
...     { course: "MERN Stack" },
...     { $set: { status: "in progress" } }
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 3,
  modifiedCount: 3,
  upsertedCount: 0
}
studentDB> db.students.find()
[
  {
    _id: ObjectId('6ae4b87f3d4d3f93c687985b'),
    name: 'Arun',
    age: 21,
    course: 'MERN Stack',
    status: 'in progress'
  },
  {
    _id: ObjectId('6ae4b87f3d4d3f93c687985d'),
    name: 'Jefferson',
    age: 21,
    course: 'MERN Stack',
    status: 'in progress'
  },
  {
    _id: ObjectId('6ae4b87f3d4d3f93c687985c'),
    name: 'Priya',
    age: 22,
    course: 'Python',
    status: 'completed'
  },
  {
    _id: ObjectId('6ae4b87f3d4d3f93c687985e'),
    name: 'Rahul',
    age: 19,
```

Deleteone and deleteMany( ):

```
studentDB> db.students.deleteOne({
...     name: "Divya"
... })
{ acknowledged: true, deletedCount: 1 }
studentDB> db.students.find()
[
  {
    _id: ObjectId('6ae4407f3d4d3f93c687065f'),
    name: 'Arun',
    age: 21,
    course: 'MERN Stack',
    status: 'in progress'
  },
  {
    _id: ObjectId('6ae4407f3d4d3f93c6870660'),
    name: 'Jefferson',
    age: 21,
    course: 'MERN Stack',
    status: 'in progress'
  },
  {
    _id: ObjectId('6ae4407f3d4d3f93c6870659'),
    name: 'Priya',
    age: 22,
    course: 'Python',
    status: 'completed'
  },
  {
    _id: ObjectId('6ae4407f3d4d3f93c687065a'),
    name: 'Rahul',
    age: 19,
    course: 'MERN Stack',
    status: 'in progress'
  }
]
studentDB> db.students.deleteMany({})
{ acknowledged: true, deletedCount: 4 }
studentDB> db.students.find()

studentDB> show collections
students
studentDB>
```

MongoDB Query Operator

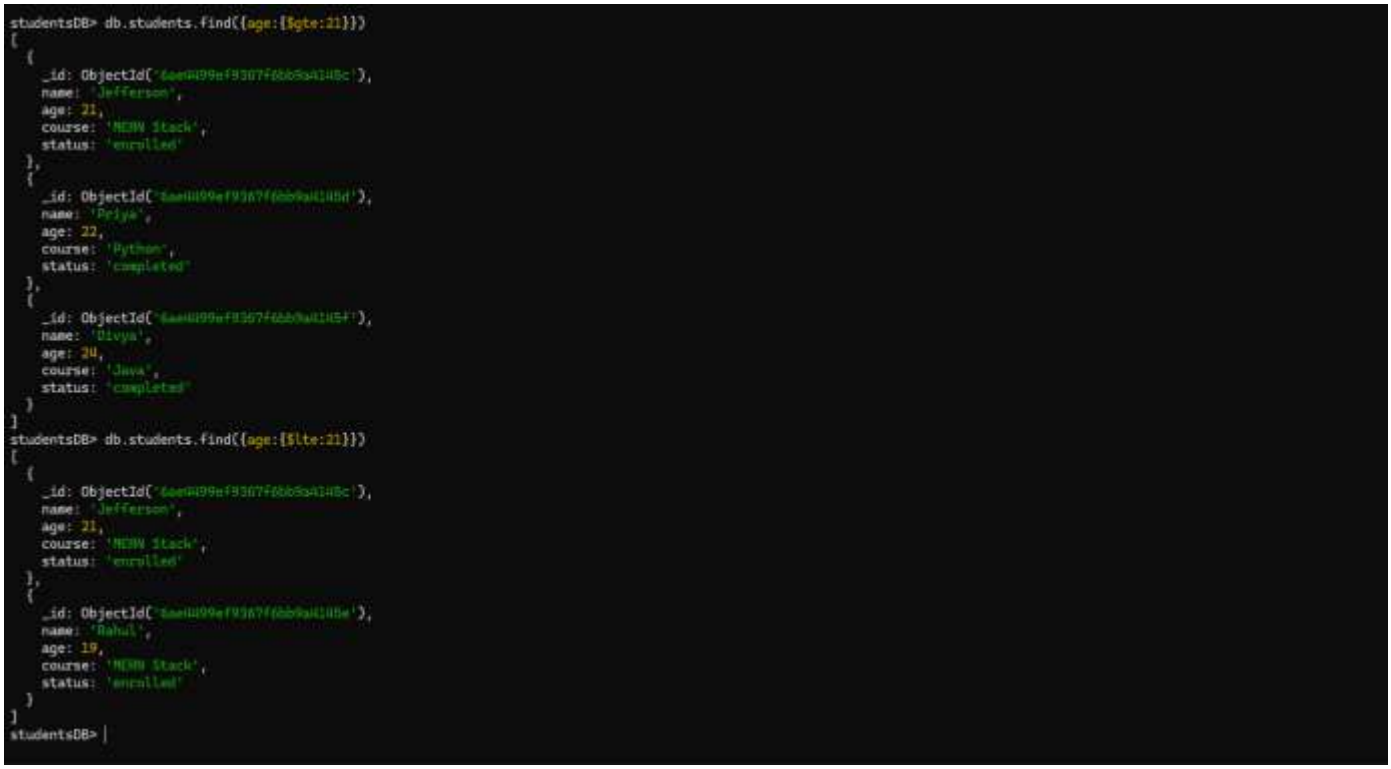
1. $gt and $lt – Greater Than and Less Than

```
studentsDB> db.students.find({age: {$lt:20}})
[
  {
    _id: ObjectId('6ae4499ef9367f6bb9a0145e'),
    name: 'Rahul',
    age: 19,
    course: 'MERN Stack',
    status: 'enrolled'
  }
]
studentsDB> db.students.find(age:{$gt:20}})
Uncaught:
SyntaxError: Unexpected token, expected "," (1:20)

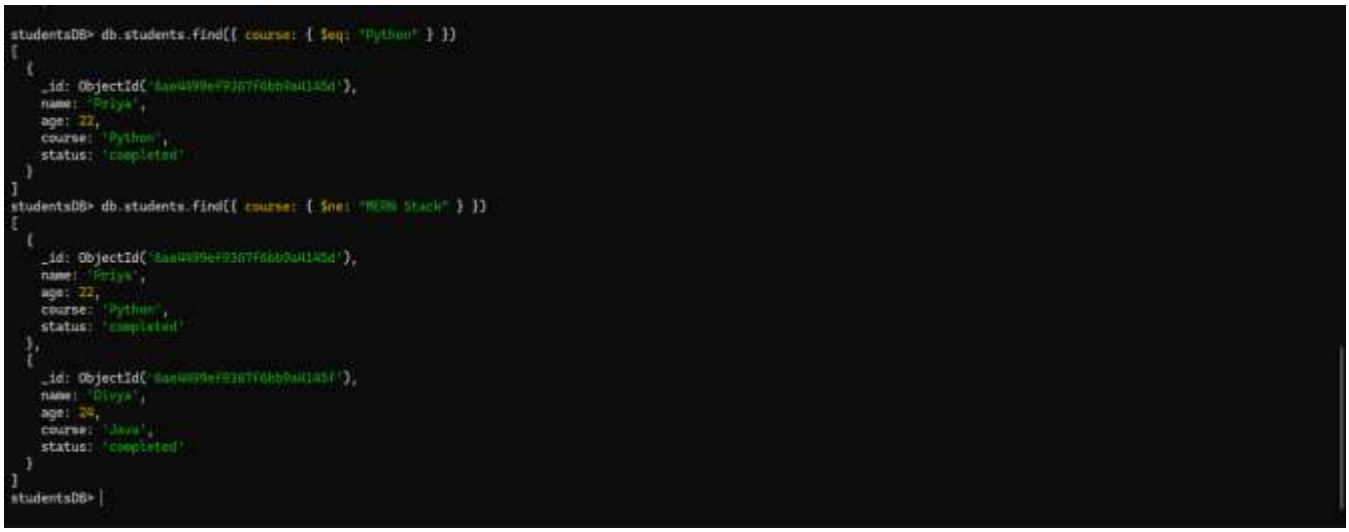
> 1 | db.students.find(age:{$gt:20}})
    |                     ^
  2 |

studentsDB> db.students.find({age:{$gt:20}})
[
  {
    _id: ObjectId('6ae4499ef9367f6bb9a0145c'),
    name: 'Jefferson',
    age: 21,
    course: 'MERN Stack',
    status: 'enrolled'
  },
  {
    _id: ObjectId('6ae4499ef9367f6bb9a0145d'),
    name: 'Priya',
    age: 22,
    course: 'Python',
    status: 'completed'
  },
  {
    _id: ObjectId('6ae4499ef9367f6bb9a0145f'),
    name: 'Divya',
    age: 26,
    course: 'Java',
    status: 'completed'
  }
]
studentsDB>
```

2. $gte and $lte – Greater Than or Equal To and Less Than or Equal To



3. $eq and $ne – Equal To and Not Equal To



4. $in and $nin – In and Not In

```
]
studentsDB> db.students.find({
|   course: { $in: ["MERN Stack", "Python"] }
| })
[
  {
    _id: ObjectId('6ae4499ef9367f6bb9a4145c'),
    name: 'Jefferson',
    age: 21,
    course: 'MERN Stack',
    status: 'enrolled'
  },
  {
    _id: ObjectId('6ae4499ef9367f6bb9a4145d'),
    name: 'Priya',
    age: 22,
    course: 'Python',
    status: 'completed'
  },
  {
    _id: ObjectId('6ae4499ef9367f6bb9a4145e'),
    name: 'Rahul',
    age: 19,
    course: 'MERN Stack',
    status: 'enrolled'
  }
]
studentsDB> db.students.find({
|   course: { $nin: ["MERN Stack", "Python"] }
| })
[
  {
    _id: ObjectId('6ae4499ef9367f6bb9a4145f'),
    name: 'Divya',
    age: 24,
    course: 'Java',
    status: 'completed'
  }
]
studentsDB>
```

5. $and and $or – AND and OR Conditions

```
studentsDB> db.students.find({$or:[ {age:{$eq:22}},{course:"python"}]})
[
  {
    _id: ObjectId('6ae4499ef9367f6bb9a4145d'),
    name: 'Priya',
    age: 22,
    course: 'Python',
    status: 'completed'
  }
]
studentsDB> studentsDB> db.students.find({$and:[{age:{$gt:20}},{status:"enrolled"}]})
ReferenceError: studentsDB is not defined
studentsDB> db.students.find({$and:[{age:{$gt:20}},{status:"enrolled"}]})
[
  {
    _id: ObjectId('6ae4499ef9367f6bb9a4145c'),
    name: 'Jefferson',
    age: 21,
    course: 'MERN Stack',
    status: 'enrolled'
  }
]
studentsDB>
```

6. $expr – Expression Operator

```
studentDB> db.students.updateOne(
|   { name: "Arun" },
|   { $set: { marks: 85, passMark: 40 } }
| )
|
| db.students.updateOne(
|   { name: "Jefferson" },
|   { $set: { marks: 35, passMark: 40 } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
studentDB> db.students.find({
|   $expr: { $gt: ["$marks", "$passMark"] }
| })
[
  {
    _id: ObjectId('6ae4bb3bf9367f6bb9a41460'),
    name: 'Arun',
    age: 21,
    course: 'MERN Stack',
    status: 'enrolled',
    marks: 85,
    passMark: 40
  }
]
studentDB>
```

7. $not and $exists – NOT Condition and Field Existence



```
studentsDB> db.students.find({
|   age: { $not: { $gt: 21 } }
| })
[
  {
    _id: ObjectId('6aee4099ef93d7f6bb9a41d5c'),
    name: 'Jefferson',
    age: 21,
    course: 'MERN Stack',
    status: 'enrolled'
  },
  {
    _id: ObjectId('6aee4099ef93d7f6bb9a41d5e'),
    name: 'Rahul',
    age: 19,
    course: 'MERN Stack',
    status: 'enrolled'
  }
]
studentsDB> db.students.find({
|   status: { $exists: true }
| })
[
  {
    _id: ObjectId('6aee4099ef93d7f6bb9a41d5c'),
    name: 'Jefferson',
    age: 21,
    course: 'MERN Stack',
    status: 'enrolled'
  },
  {
    _id: ObjectId('6aee4099ef93d7f6bb9a41d5d'),
    name: 'Priya',
    age: 22,
    course: 'Python',
    status: 'completed'
  },
  {
    _id: ObjectId('6aee4099ef93d7f6bb9a41d5e'),
    name: 'Rahul',
    age: 19,
    course: 'MERN Stack',
    status: 'enrolled'
  },
```

REAL WORLD USE CASE

LIBRARY MANAGEMENT SYSTEM

```
test> use libraryDB
switched to db libraryDB
libraryDB> db.createCollection("books")
{ ok: 1 }
libraryDB> show collections
books
libraryDB> db.books.insertMany([
...   {
...     title: "The Alchemist",
...     author: "Paulo Coelho",
...     category: "Fiction",
...     available: true
...   },
...   {
...     title: "Wings of Fire",
...     author: "A. P. J. Abdul Kalam",
...     category: "Biography",
...     available: true
...   },
...   {
...     title: "Clean Code",
...     author: "Robert C. Martin",
...     category: "Programming",
...     available: false
...   }
... ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ae45bfbd7e56e13b27cffac'),
    '1': ObjectId('6ae45bfbd7e56e13b27cffad'),
    '2': ObjectId('6ae45bfbd7e56e13b27cffae')
  }
}
libraryDB> db.books.find()
[
  {
    _id: ObjectId('6ae45bfbd7e56e13b27cffac'),
    title: 'The Alchemist',
    author: 'Paulo Coelho',
    category: 'Fiction',
    available: true
  },
```

```
]
libraryDB> db.books.updateOne(
...   { title: "The Alchemist" },
...   { $set: { available: false } }
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
libraryDB> db.books.find()
[
  {
    _id: ObjectId('6ae45bfbd7e56e13b27cffac'),
    title: 'The Alchemist',
    author: 'Paulo Coelho',
    category: 'Fiction',
    available: false
  },
  {
    _id: ObjectId('6ae45bfbd7e56e13b27cffad'),
    title: 'Wings of Fire',
    author: 'A. P. J. Abdul Kalam',
    category: 'Biography',
    available: true
  },
  {
    _id: ObjectId('6ae45bfbd7e56e13b27cffae'),
    title: 'Clean Code',
    author: 'Robert C. Martin',
    category: 'Programming',
    available: false
  }
]
libraryDB>
```

MongoDB Practical – Theory and Explanation

MongoDB Practical Assignment

1. Introduction :

MongoDB is a NoSQL database used to store and manage data in the form of documents. It stores data in a flexible, JSON - like format.

2. Database and Collections :

In this practical, I created a database named StudentDB and collection named students. I also created another database named LibraryDB with a collection named books for library management.

3. CRUD Operations :

- Create : insert single and multiple student record using insertOne() and insertMany().
- Read : Retrived all records using find().
- update : Modified student details using update().
- Delete : Deleting a Records using deleteOne() and many()

iv. Query Operators :

- $gt & $lt - Greater than, Less than
- $gte & $lte - Greater than or equal to and Less than
- $eq & $ne - Equal to & not Equal to
- $in - Matches any value in a list
- $nin - Does not match value in a list
- $and - check wheather all conditions are true
- $or - checks wheather at least one condition true
- $not - Negates a Conditions
- $exists - check wheather a field exists

v. Real - World Use Case - Library Management system

I implemented a simple library management system using MongoDB. I Created a library DB database and books Collection.

I inserted book details such as title, author, Category and availablity status. I retrived available books using queries.